

Uniwersytet w Białymstoku

Wydział Informatyki

Kierunek: Informatyka II stopnia
Rok I semestr I

Inteligentne multimedialne systemy uczące

IMSU

Prowadzący:
dr inż. Wiesław Półjanowicz

Wykonali:
Patryk Kulesza
Bartłomiej Koźluk
Mateusz Malewski

Białystok, dn. .....

Spis treści

1 Treść zadania projektowego **3**

2 Cel budowania systemu i zakres systemu **3**
2.1 Cele główne . . . 3
2.2 Cele szczegółowe . . . 3
2.3 Zakres systemu . . . 4

3 Perspektywa wariantów użycia systemu **5**
3.1 Diagram wariantów użycia systemu . . . 5
3.2 Opisy tekstowe aktorów . . . 5
3.3 Opisy tekstowe wariantów użycia . . . 6
3.3.1 Rejestracja . . . 6
3.3.2 Logowanie . . . 6
3.3.3 Wypełnienie ankiety VAK . . . 6
3.3.4 Przeglądanie kursów i klas . . . 7
3.3.5 Nauka z lekcji . . . 7
3.3.6 Rozwiązanie quizu . . . 7
3.3.7 Zarządzanie stylem . . . 8
3.3.8 Zarządzanie kursami . . . 8
3.3.9 Edycja lekcji . . . 8
3.3.10 Edycja quizu . . . 9
3.3.11 Zarządzanie klasami . . . 9
3.3.12 Zarządzanie kontami . . . 9
3.4 Przykładowe diagramy sekwencji . . . 10
3.4.1 Przykład 1. Pierwsze logowanie ucznia i ankieta VAK . . . 10
3.4.2 Przykład 2. Lekcja dopasowana do stylu i rozwinięcie materiału . . . 11
3.4.3 Przykład 3. Rozwiązanie quizu i adaptacja . . . 11

4 Perspektywa logiczna **12**
4.1 Diagram klas . . . 12
4.2 Uporządkowany alfabetycznie wykaz klas . . . 12
4.3 Model danych . . . 13
4.4 Reguły adaptacji . . . 14
4.4.1 Styl i wagi z ankiety . . . 14
4.4.2 Podanie lekcji . . . 14
4.4.3 Styl efektywny lekcji . . . 14
4.4.4 Aktualizacja wag po quizie . . . 14
4.4.5 Propozycja zmiany stylu . . . 14
4.4.6 Przykład . . . 15

5 Technologie **15**
5.1 Architektura . . . 15
5.2 Technologie oprogramowania . . . 15
5.3 Bezpieczeństwo . . . 16
5.4 Licencje . . . 17

6 Dokumentacja testów **17**
6.1 Cel i zakres testowania . . . 17
6.2 Strategia testowania . . . 17
6.3 Środowisko testowe . . . 18
6.4 Przypadki testowe . . . 18
6.5 Scenariusze testowe . . . 19
6.5.1 ST-01: Od ankiety do adaptacji . . . 19
6.5.2 ST-02: Propozycja zmiany stylu . . . 19
6.5.3 ST-03: Dostęp do kursów przez klasy . . . 19
6.6 Podsumowanie pokrycia testami . . . 20

7 Instrukcja uruchomienia aplikacji lokalnie **20**
7.1 Wymagania . . . 20
7.2 Krok 1: Pobranie repozytorium . . . 20
7.3 Krok 2: Zmienne środowiskowe . . . 21
7.4 Krok 3: Baza danych . . . 21
7.5 Krok 4: Serwer . . . 21
7.6 Krok 5: Klient . . . 22
7.7 Krok 6: Weryfikacja . . . 22
7.8 Uruchomienie testów i kontrola kodu . . . 23
7.9 Rozwiązywanie problemów . . . 23
7.9.1 `/health` zwraca `"database": "error"` . . . 23
7.9.2 Serwer nie startuje: błąd `jwt_secret` . . . 23
7.9.3 Port 3001, 8001 lub 5433 jest zajęty . . . 23
7.9.4 Brak pytań ankiety lub kursów demonstracyjnych . . . 23
7.10 Struktura projektu . . . 23
7.11 Przydatne polecenia . . . 24

1 Treść zadania projektowego

IMSU – multimedialny inteligentny system uczący:

Klasyczne kursy internetowe podają materiał każdemu uczniowi w ten sam sposób, choć jedni lepiej uczą się z rysunków i schematów, inni z nagrań i rozmowy, a jeszcze inni przez samodzielne wykonywanie ćwiczeń. Nauczyciel prowadzący zajęcia na żywo dostosowuje się do ucznia odruchowo – system internetowy musi to robić według jawnych reguł.

Celem projektu jest system, w którym **nauczyciel** składa w edytorze kurs z lekcji zawierających tekst oraz materiały multimedialne (obrazy, filmy, nagrania, karty PDF, linki), oznaczając każdy materiał formą przekazu: wzrokową (V), słuchową (A) lub kinestetyczną (K). **Uczeń** po rejestracji wypełnia ankietę VAK, z której system wyznacza jego styl uczenia się oraz wagi poszczególnych form. Na tej podstawie system decyduje, jak podać lekcję: materiały pasujące do stylu ucznia są rozwinięte, pozostałe zwinięte, ale dostępne. Po każdym quizie system aktualizuje wagi form na podstawie wyniku i materiałów, z których uczeń faktycznie korzystał, a gdy inna forma wyraźnie wypada lepiej – proponuje zmianę stylu wraz z uzasadnieniem. Ostateczną decyzję podejmuje uczeń.

System obsługuje trzy role: ucznia, nauczyciela i administratora, który zarządza kontami użytkowników.

2 Cel budowania systemu i zakres systemu

2.1 Cele główne

- Dopasowanie sposobu podania materiału do stylu uczenia się ucznia (VAK)
- Udostępnienie nauczycielowi edytora kursów z materiałami oznaczonymi formą V, A lub K
- Aktualizacja modelu ucznia na podstawie wyników quizów i proponowanie zmiany stylu
- Wyjaśnialność: każda decyzja systemu ma widoczne uzasadnienie („dlaczego to widzisz")

2.2 Cele szczegółowe

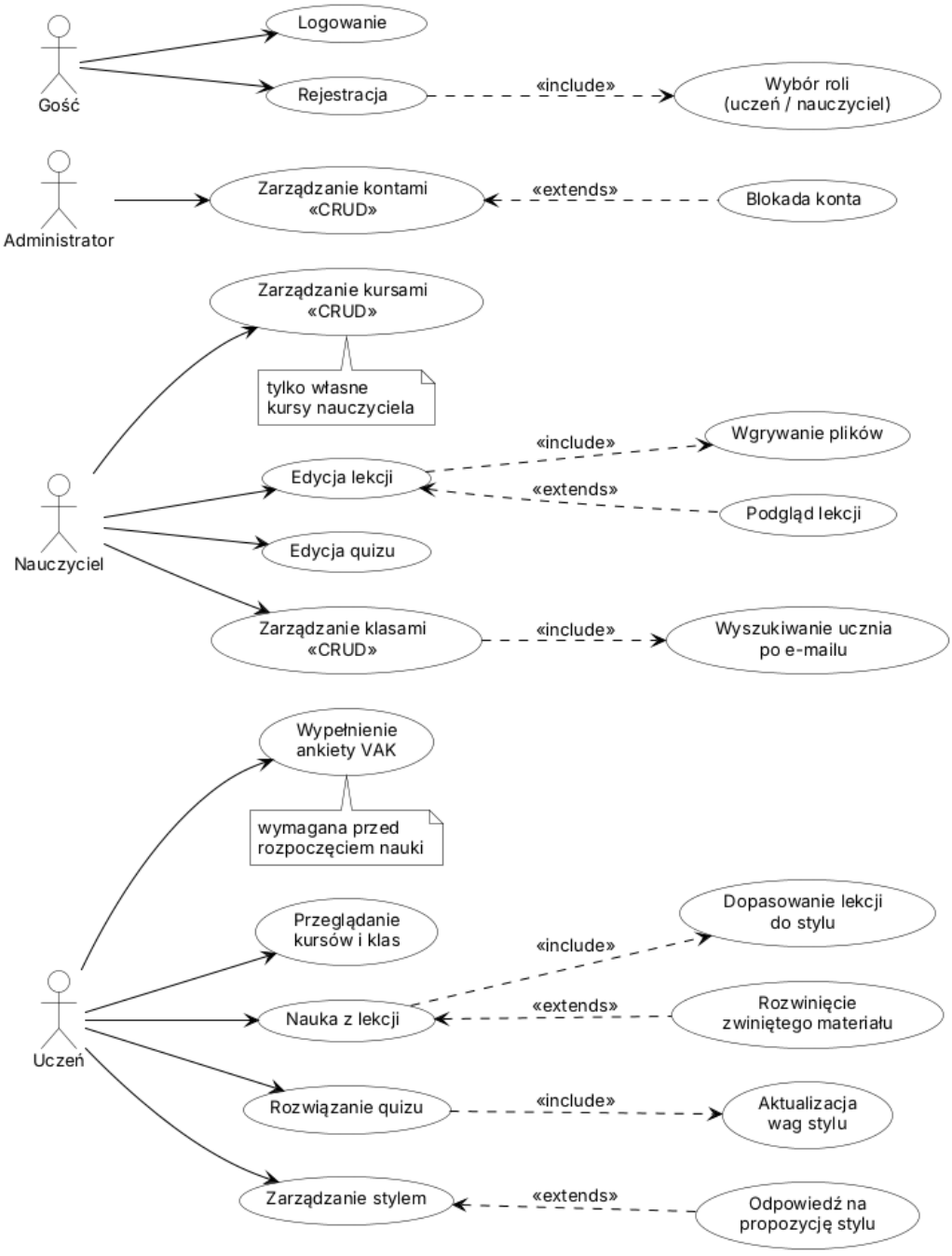
- Wyznaczenie stylu ucznia z ankiety VAK jako punktu startowego, z możliwością ręcznej zmiany
- Zachowanie dostępu do wszystkich materiałów – materiały innych stylów są zwinięte, nie ukryte
- Dobór pytań quizu do materiałów, z których uczeń korzystał
- Ocenianie quizów wyłącznie po stronie serwera, odporne na wielokrotne podejścia
- Udostępnianie kursów uczniom przez katalog kursów opublikowanych oraz przez klasy nauczyciela
- Prosty i czytelny interfejs, przystosowany do młodszych użytkowników

2.3 Zakres systemu

- Rejestracja z wyborem roli (uczeń lub nauczyciel), logowanie, sesja
- Administracja kont nauczycieli i uczniów (tworzenie, edycja, blokada)
- Kursy i lekcje: tworzenie, kolejność, publikacja, archiwizacja
- Edytor lekcji: bloki tekstu (formatowanie, tabele, wzory) i załączniki z formami V/A/K
- Quiz lekcji z pytaniami wspólnymi i pytaniami o materiały danej formy
- Ankieta VAK, profil ucznia (styl, wagi, historia), propozycja zmiany stylu
- Klasy nauczyciela: uczniowie i przypisane kursy

3 Perspektywa wariantów użycia systemu

3.1 Diagram wariantów użycia systemu



Rysunek 1: Diagram przypadków użycia systemu

3.2 Opisy tekstowe aktorów

- **Gość** – osoba niezalogowana. Widzi stronę startową z opisem systemu, może założyć konto (jako uczeń albo nauczyciel) lub zalogować się.
- **Uczeń** – zalogowany użytkownik, który się uczy. Przed rozpoczęciem nauki musi wypeł-

nić ankietę VAK. Ma dostęp do kursów opublikowanych oraz kursów przypisanych do jego klas, uczy się z lekcji dopasowanych do swojego stylu, rozwiązuje quizy i zarządza swoim stylem uczenia się.

- **Nauczyciel** – tworzy i edytuje własne kursy, lekcje, materiały i quizy, publikuje kursy oraz prowadzi klasy, do których dodaje uczniów i przypisuje kursy.
- **Administrator** – zarządza kontami nauczycieli i uczniów: tworzy je, edytuje i blokuje. Konto administratora tworzy się poleceniem w konsoli, nie przez rejestrację.

3.3 Opisy tekstowe wariantów użycia

3.3.1 Rejestracja

Numer: UC-01 **Aktor:** Gość

Krótki opis: Założenie konta ucznia albo nauczyciela. Rola jest wybierana przy rejestracji i nie zmienia się później.

Główny przepływ zdarzeń:

1. Gość klika „Załóż konto” na stronie startowej
2. Gość wybiera rolę (Uczeń / Nauczyciel), podaje imię, e-mail i hasło (min. 8 znaków)
3. System sprawdza dane i unikalność adresu e-mail
4. System tworzy konto, rozpoczyna sesję i przekierowuje do panelu użytkownika

3.3.2 Logowanie

Numer: UC-02 **Aktor:** Gość

Krótki opis: Zalogowanie się adresem e-mail i hasłem.

Główny przepływ zdarzeń:

1. Gość klika „Zaloguj się” i podaje e-mail oraz hasło
2. System weryfikuje dane oraz to, czy konto nie jest zablokowane
3. System zapisuje sesję w ciasteczku i przekierowuje do panelu użytkownika

3.3.3 Wypełnienie ankiety VAK

Numer: UC-03 **Aktor:** Uczeń

Krótki opis: Ankieta wyznacza styl uczenia się ucznia i startowe wagi form V, A, K. Uczeń bez ankiety jest na nią kierowany z każdej strony systemu.

Główny przepływ zdarzeń:

1. System przekierowuje ucznia bez ankiety na stronę ankiety
2. Uczeń odpowiada na 20 pytań (każda odpowiedź odpowiada jednej formie)
3. System zlicza punkty V, A, K i wyznacza styl (forma lub formy z największą liczbą punktów)
4. System zapisuje wynik oraz profil ucznia i wyświetla styl z punktami na wykresie

3.3.4 Przeglądanie kursów i klas

Numer: UC-04 **Aktor:** Uczeń

Krótki opis: Uczeń przegląda katalog kursów opublikowanych oraz swoje klasy z przypisanymi kursami.

Główny przepływ zdarzeń:

1. Uczeń wybiera „Kursy” – system wyświetla kursy opublikowane
2. Uczeń wybiera „Moje klasy” – system wyświetla klasy ucznia z nauczycielem i liczbą kursów
3. Uczeń otwiera kurs – system wyświetla wstęp i listę lekcji

3.3.5 Nauka z lekcji

Numer: UC-05 **Aktor:** Uczeń

Krótki opis: Uczeń otwiera lekcję podaną według swojego stylu i w razie potrzeby rozwija materiały innych stylów.

Główny przepływ zdarzeń:

1. Uczeń otwiera lekcję z listy lekcji kursu
2. System wyświetla bloki w kolejności nauczyciela; załączniki form spoza stylu ucznia są zwinięte
3. System wyświetla uzasadnienie „dlaczego to widzisz” i liczbę zwiniętych materiałów
4. Uczeń może kliknąć „Pokaż materiał” – system rozwija materiał i zapamiętuje jego formy w stylu efektywnym lekcji

3.3.6 Rozwiązanie quizu

Numer: UC-06 **Aktor:** Uczeń

Krótki opis: Uczeń rozwiązuje quiz na końcu lekcji; system ocenia wynik i aktualizuje wagi stylu (<<include>> Aktualizacja wag stylu).

Główny przepływ zdarzeń:

1. System wyświetla pytania wspólne i pytania o materiały form stylu efektywnego
2. Uczeń zaznacza odpowiedzi i klika „Sprawdź odpowiedzi”
3. System ocenia odpowiedzi; wynik co najmniej 70% oznacza zaliczenie
4. System aktualizuje wagi form (pierwszy sukces i pierwsza porażka w lekcji)
5. System wyświetla poprawne odpowiedzi, wynik i informację o zmianie wag

3.3.7 Zarządzanie stylem

Numer: UC-07 **Aktor:** Uczeń

Krótki opis: Uczeń przegląda swój styl i wagi, zmienia styl ręcznie lub odpowiada na propozycję zmiany stylu.

Główny przepływ zdarzeń:

1. Uczeń wybiera „Mój styl” – system wyświetla styl, jego źródło, wagi i historię ankiet
2. Jeśli system ma propozycję, wyświetla baner z uzasadnieniem
3. Uczeń klika „Przełącz” (styl zmienia się na proponowany) albo „Zostaję przy swoim”
4. Uczeń może też ręcznie wybrać styl, wrócić do stylu z ankiety lub wypełnić ankietę ponownie

3.3.8 Zarządzanie kursami

Numer: UC-08 **Aktor:** Nauczyciel

Krótki opis: Nauczyciel tworzy i edytuje swoje kursy i lekcje oraz decyduje o publikacji.

Główny przepływ zdarzeń:

1. Nauczyciel wybiera „Moje kursy” i klika „Nowy kurs”
2. Nauczyciel podaje tytuł i opis; system tworzy kurs prywatny
3. Nauczyciel uzupełnia wstęp, dodaje lekcje i ustala ich kolejność
4. Nauczyciel publikuje kurs – kurs pojawia się w katalogu uczniów

3.3.9 Edycja lekcji

Numer: UC-09 **Aktor:** Nauczyciel

Krótki opis: Nauczyciel układa treść lekcji z bloków tekstu i załączników oznaczonych formami V, A, K.

Główny przepływ zdarzeń:

1. Nauczyciel otwiera lekcję w edytorze
2. Nauczyciel dodaje blok tekstu („+ Tekst”) lub załącznik („+ Załącznik”)
3. Przy załączniku wybiera typ, wgrywa plik lub podaje adres, wpisuje tytuł i zaznacza formy V/A/K
4. Nauczyciel zmienia kolejność bloków i klika „Zapisz lekcję”
5. System sprawdza poprawność bloków i plików oraz zapisuje lekcję

3.3.10 Edycja quizu

Numer: UC-10 **Aktor:** Nauczyciel
Krótki opis: Nauczyciel tworzy quiz na końcu lekcji.
Główny przepływ zdarzeń:

1. Nauczyciel w podglądzie lekcji klika „Dodaj quiz” lub „Edytuj quiz”
2. Nauczyciel dodaje pytania (3–4 odpowiedzi, jedna poprawna) i wskazuje, czy pytanie dotyczy materiału formy V, A lub K
3. Nauczyciel klika „Zapisz quiz”; system sprawdza i zapisuje quiz

3.3.11 Zarządzanie klasami

Numer: UC-11 **Aktor:** Nauczyciel
Krótki opis: Nauczyciel tworzy klasy, dodaje do nich uczniów i przypisuje kursy (także prywatne).
Główny przepływ zdarzeń:

1. Nauczyciel wybiera „Klasy” i tworzy nową klasę
2. Nauczyciel klika „Dodaj ucznia”, wpisuje fragment e-maila i dodaje ucznia z wyników wyszukiwania
3. Nauczyciel klika „Przypisz kurs” i wybiera swój kurs
4. Uczniowie klasy widzą przypisane kursy w „Moich klasach”

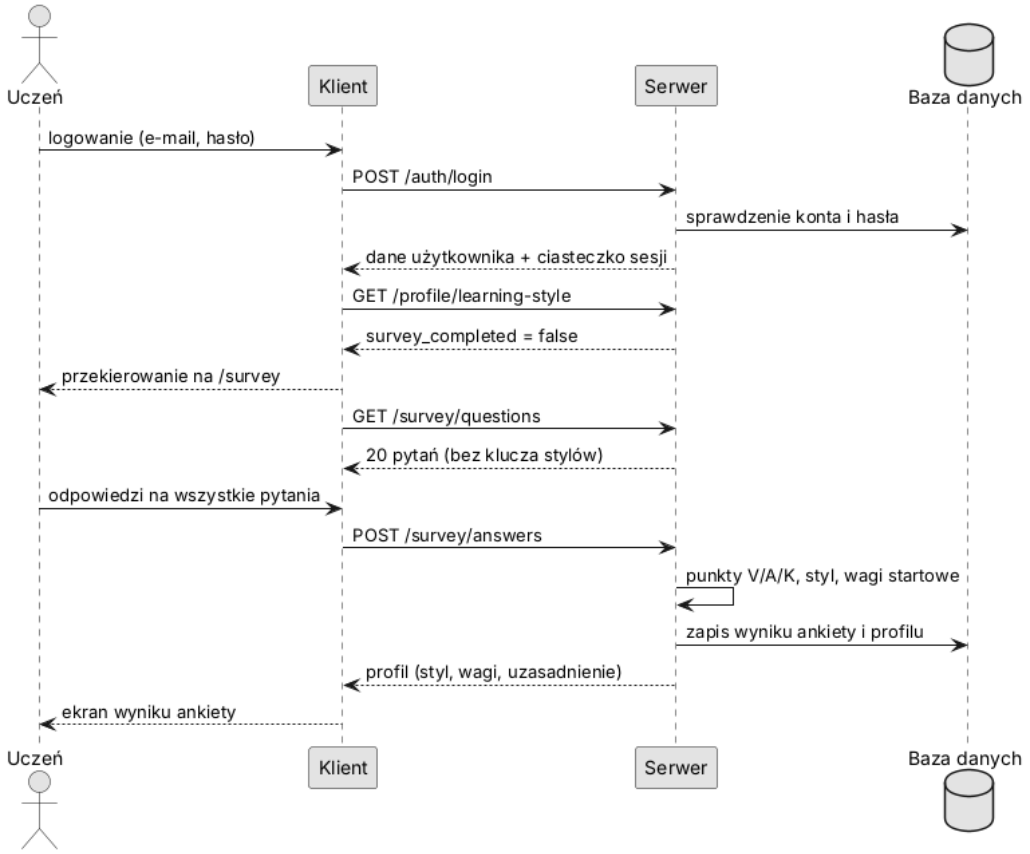
3.3.12 Zarządzanie kontami

Numer: UC-12 **Aktor:** Administrator
Krótki opis: Administrator zarządza kontami nauczycieli i uczniów.
Główny przepływ zdarzeń:

1. Administrator wybiera „Użytkownicy” i zakładkę Nauczyciele lub Uczniowie
2. Administrator dodaje konto (imię, e-mail, hasło) lub edytuje istniejące
3. Administrator może zablokować konto – zablokowany użytkownik nie zaloguje się

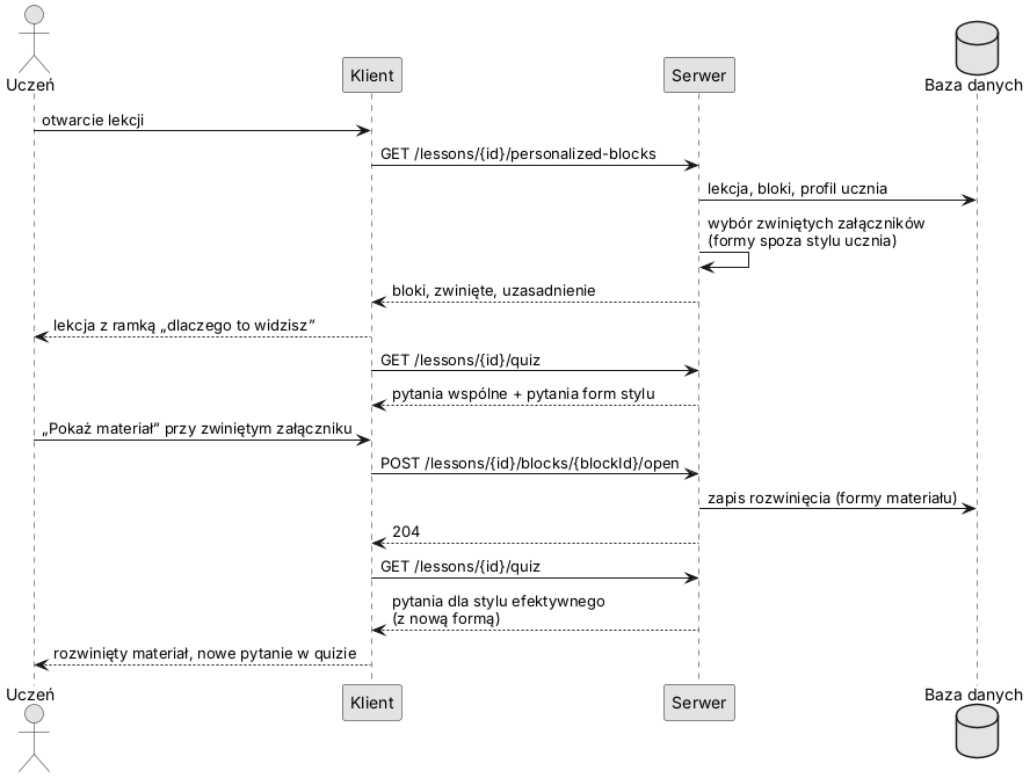
3.4 Przykładowe diagramy sekwencji

3.4.1 Przykład 1. Pierwsze logowanie ucznia i ankieta VAK



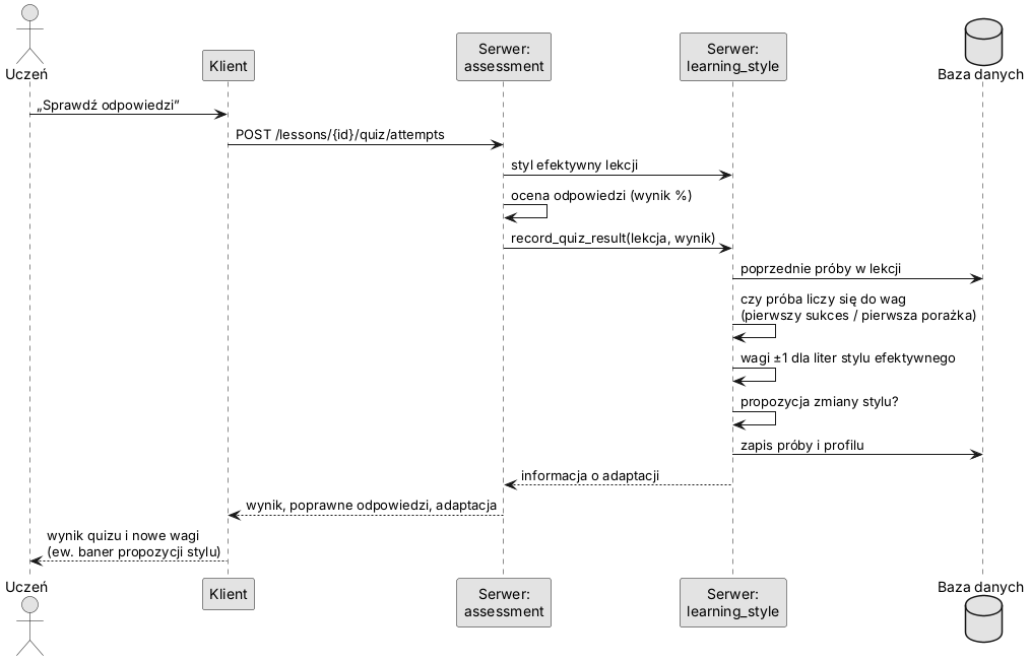
Uczeń bez wypełnionej ankiety loguje się do systemu. Klient pobiera profil ucznia, a ponieważ ankieta nie została wypełniona, przekierowuje ucznia na stronę ankiety. Pytania przychodzą bez informacji, której formie odpowiada dana odpowiedź – punkty liczy serwer. Po wysłaniu odpowiedzi serwer wyznacza styl i wagi startowe, zapisuje wynik i profil, a uczeń widzi ekran z wynikiem.

3.4.2 Przykład 2. Lekcja dopasowana do stylu i rozwinięcie materiału



Serwer zwraca wszystkie bloki lekcji wraz z listą załączników do zwinięcia (tych, których żadna forma nie należy do stylu ucznia) oraz uzasadnieniem. Gdy uczeń rozwija zwinięty materiał, serwer zapisuje to zdarzenie, a formy materiału trafiają do stylu efektywnego lekcji. Klient pobiera quiz ponownie – pojawia się w nim pytanie o materiał nowej formy.

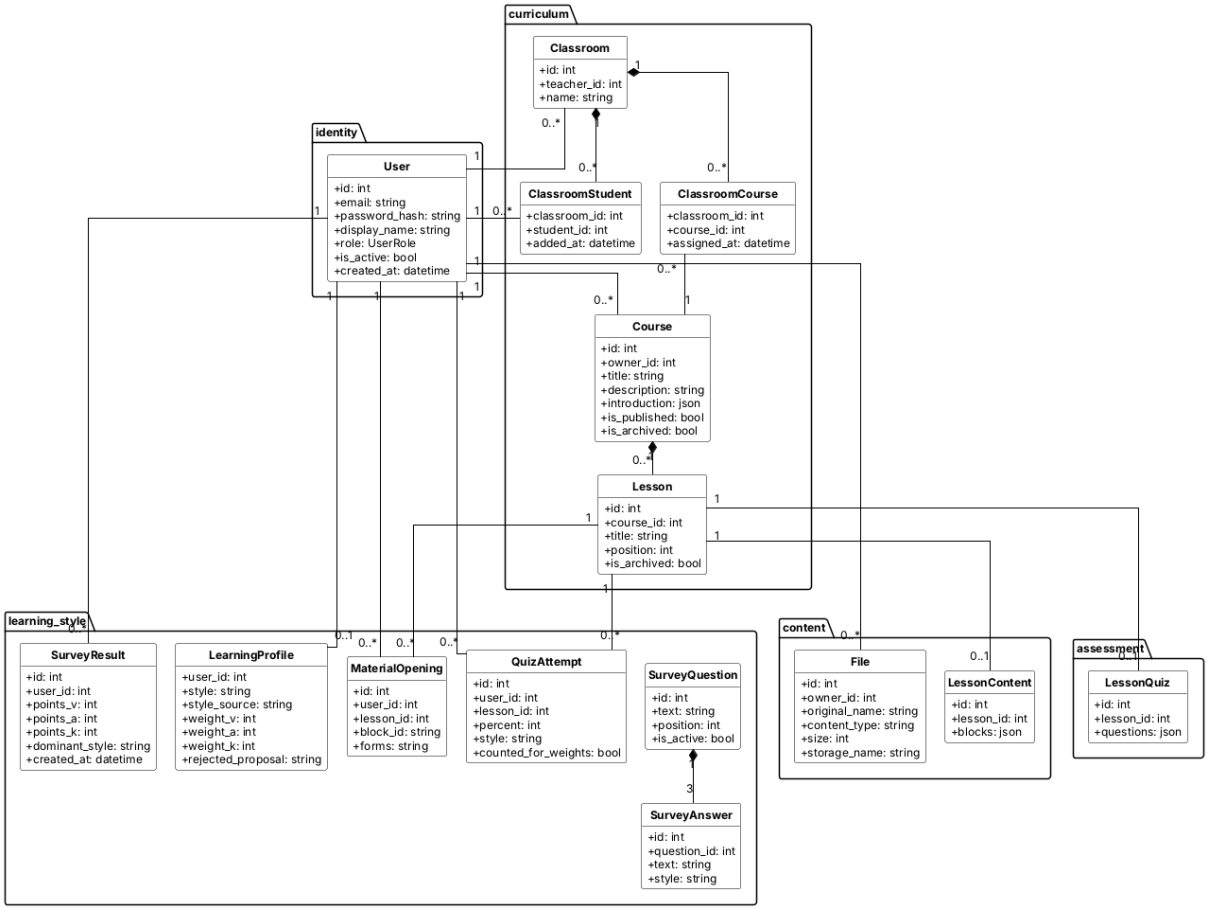
3.4.3 Przykład 3. Rozwiązanie quizu i adaptacja



Moduł `assessment` ocenia odpowiedzi wyłącznie po stronie serwera i przekazuje wynik do modułu `learning_style` przez jedną funkcję `record_quiz_result`. Moduł stylu sprawdza, czy próba liczy się do wag, zmienia wagi liter stylu efektywnego o ± 1 i sprawdza, czy należy zaproponować zmianę stylu. Uczeń widzi wynik, poprawne odpowiedzi i informację o adaptacji.

4 Perspektywa logiczna

4.1 Diagram klas



Rysunek 2: Diagram klas (klasy pogrupowane według modułów systemu)

4.2 Uporządkowany alfabetycznie wykaz klas

Classroom – klasa (grupa uczniów) prowadzona przez jednego nauczyciela.
Atrybuty: `id`, `teacher_id`, `name`.

ClassroomCourse – przypisanie kursu do klasy; uczniowie klasy widzą przypisany kurs, także prywatny.
Atrybuty: `classroom_id`, `course_id`, `assigned_at`.

ClassroomStudent – przynależność ucznia do klasy.
Atrybuty: `classroom_id`, `student_id`, `added_at`.

Course – kurs nauczyciela: opis widoczny w katalogu, wstęp i lista lekcji. Kurs jest prywatny albo opublikowany; usunięcie oznacza archiwizację.
Atrybuty: `id`, `owner_id`, `title`, `description`, `introduction` (dokument edytora), `is_published`, `is_archived`.

File – plik wgrany przez nauczyciela (obraz, nagranie, film, PDF), przechowywany na dysku pod losową nazwą.
Atrybuty: `id`, `owner_id`, `original_name`, `content_type`, `size`, `storage_name`.

LearningProfile – profil ucznia: aktualny styl (np. `V`, `VA`), źródło stylu (ankieta, wybór ręczny, adaptacja), wagi form V, A, K (0–30) oraz odrzucona propozycja zmiany stylu.
Atrybuty: `user_id`, `style`, `style_source`, `weight_v`, `weight_a`, `weight_k`, `rejected_proposal`.

Lesson – lekcja kursu w kolejności ustalonej przez nauczyciela.
Atrybuty: `id`, `course_id`, `title`, `position`, `is_archived`.

LessonContent – treść lekcji jako uporządkowana lista bloków: tekstu oraz załączników z formami V/A/K.
Atrybuty: `id`, `lesson_id`, `blocks` (JSON).

LessonQuiz – quiz na końcu lekcji: do 20 pytań jednokrotnego wyboru, każde wspólne albo dotyczące materiału formy V, A lub K.
Atrybuty: `id`, `lesson_id`, `questions` (JSON).

MaterialOpening – zapis pierwszego rozwinięcia zwiniętego materiału przez ucznia; jego formy wchodzą do stylu efektywnego lekcji.
Atrybuty: `id`, `user_id`, `lesson_id`, `block_id`, `forms`.

QuizAttempt – podejście ucznia do quizu: wynik w procentach, styl efektywny w chwili podejścia i informacja, czy podejście zmieniło wagi.
Atrybuty: `id`, `user_id`, `lesson_id`, `percent`, `style`, `counted_for_weights`.

SurveyAnswer – odpowiedź na pytanie ankiety, przypisana do jednej formy V, A lub K.
Atrybuty: `id`, `question_id`, `text`, `style`.

SurveyQuestion – pytanie ankiety VAK (20 pytań, po 3 odpowiedzi).
Atrybuty: `id`, `text`, `position`, `is_active`.

SurveyResult – wynik jednego wypełnienia ankiety (historia wyników).
Atrybuty: `id`, `user_id`, `points_v`, `points_a`, `points_k`, `dominant_style`, `created_at`.

User – konto użytkownika z rolą ucznia, nauczyciela albo administratora. Konto można zablokować.
Atrybuty: `id`, `email`, `password_hash`, `display_name`, `role`, `is_active`, `created_at`.

4.3 Model danych

Baza danych PostgreSQL zawiera 15 tabel odpowiadających klasom z diagramu. Najważniejsze relacje:

- `users` 1:N `courses` (autor), 1:N `classrooms` (nauczyciel), 1:1 `learning_profiles`, 1:N `survey_results`, `quiz_attempts`, `material_openings`

- `courses` 1:N `lessons`; `lessons` 1:1 `lesson_contents` i `lesson_quizzes`
- `classrooms` N:M `users` (przez `classroom_students`) oraz N:M `courses` (przez `classroom_courses`); usunięcie klasy usuwa tylko powiązania

4.4 Reguły adaptacji

Mechanizm adaptacji to zbiór reguł zapisanych jako czyste funkcje (bez bazy danych i bez frameworka), dzięki czemu są przewidywalne, wyjaśnialne i pokryte testami. Nie wymagają danych treningowych ani zewnętrznego modelu AI.

4.4.1 Styl i wagi z ankiety

Każda odpowiedź ankiety daje punkt jednej formie. Styl ucznia to forma z największą liczbą punktów; przy remisie styl jest mieszany (np. `VA`). Wagi startowe V, A, K są równe punktom z ankiety.

4.4.2 Podanie lekcji

Bloki są wyświetlane w kolejności nauczyciela, tekst zawsze. Załącznik, którego żadna forma nie należy do stylu ucznia, jest **zwinięty w miejscu** – uczeń widzi, że materiał istnieje, i może go rozwinąć. Nad lekcją system pokazuje uzasadnienie, np. „. . . bo tak wyszło w ankiecie (V 9 / A 8 / K 3)”.

4.4.3 Styl efektywny lekcji

Styl efektywny to styl ucznia uzupełniony o formy materiałów, które uczeń rozwinął w danej lekcji. Od niego zależą pytania quizu (uczeń dostaje pytania wspólne i pytania o materiały form, z których korzystał) oraz to, które wagi zmienia wynik.

4.4.4 Aktualizacja wag po quizie

- Wynik co najmniej 70% oznacza zaliczenie
- Każda litera stylu efektywnego dostaje $+1$ przy zaliczeniu albo -1 przy niezaliczeniu (zakres wag 0–30)
- W danej lekcji liczy się tylko pierwsze zaliczenie i pierwsze niezaliczenie – kolejne podejścia nie zmieniają wag, więc nie da się ich „nabić”

4.4.5 Propozycja zmiany stylu

System proponuje zmianę stylu, gdy jedna forma ma samodzielnie najwyższą wagę (bez remisu) i nie należy do stylu ucznia. Uczeń przyjmuje propozycję albo ją odrzuca – odrzucona propozycja nie wraca. Styl wybrany ręcznie wyłącza propozycje, a ponowna ankieta ustawia styl i wagi od nowa.

4.4.6 Przykład

Uczeń w ankiecie uzyskał V 9 / A 8 / K 3, więc jego styl to V. Uczy się z materiałów wzrokowych i rozwiązuje quizy w czterech lekcjach:

Zdarzenie	**Wynik**	**Zmiana**	**Wagi V / A / K**
Ankieta	–	–	9 / 8 / 3
Quiz w lekcji 1	85%	V +1	10 / 8 / 3
Quiz w lekcji 2	55%	V −1	9 / 8 / 3
Quiz w lekcji 3	40%	V −1	8 / 8 / 3
Quiz w lekcji 4	45%	V −1	7 / 8 / 3

Tabela 1: Zmiana wag ucznia o stylu V

Po czwartym quizie forma A jest jedynym liderem wag i nie należy do stylu ucznia, więc system wyświetla propozycję: „Wagi wskazują na materiały słuchowe – przełączyć?" z uzasadnieniem „Wagi stylów po quizach: V 7 / A 8 / K 3".

5 Technologie

5.1 Architektura

System jest aplikacją internetową złożoną z trzech części: klienta działającego w przeglądarce, serwera udostępniającego REST API oraz bazy danych. Serwer to **modularny monolit** – jedna aplikacja podzielona na moduły domenowe, które korzystają ze siebie wyłącznie przez publiczne funkcje serwisów:

Moduł	**Odpowiedzialność**
`identity`	konta, role, logowanie, sesja, administracja kont
`curriculum`	kursy, lekcje, publikacja, klasy i widoczność kursów
`content`	bloki lekcji (tekst, załączniki z formami V/A/K), pliki
`learning_style`	ankieta VAK, profil ucznia, personalizacja lekcji, adaptacja wag
`assessment`	quiz lekcji, ocenianie odpowiedzi, edytor quizu

Tabela 2: Moduły serwera

Reguły personalizacji, adaptacji i oceniania są czystymi funkcjami, niezależnymi od frameworka i bazy danych. Wynik quizu trafia do modelu ucznia tylko przez jedną funkcję modułu `learning_style`, a klient nie ma możliwości samodzielnego zgłoszenia wyniku.

5.2 Technologie oprogramowania

Klient:

- **Next.js 16** i **React 19** – aplikacja internetowa z routingiem opartym na katalogach
- **TypeScript** – typowany JavaScript
- **Tailwind CSS 4** i **shadcn/ui** (Base UI) – style i komponenty interfejsu

- **TanStack Query** – pobieranie i buforowanie danych z serwera
- **TipTap** – edytor tekstu lekcji (formatowanie, tabele, wklejanie z Worda)
- **KaTeX** – wyświetlanie wzorów matematycznych
- **React Hook Form**, **react-dropzone**, **Lucide** – formularze, wgrywanie plików, ikony

Serwer:

- **Python 3.12** i **FastAPI** – REST API
- **SQLAlchemy 2** i **psycopg 3** – dostęp do bazy danych
- **Alembic** – migracje schematu bazy
- **Pydantic** – walidacja danych wejściowych i wyjściowych
- **PyJWT** i **bcrypt** – sesja i haszowanie haseł

Baza danych:

- **PostgreSQL 17** uruchamiany w kontenerze **Docker** (Docker Compose); kolumny **JSONB** dla treści lekcji i quizów
- Pliki wgrane przez nauczycieli przechowywane na dysku serwera, poza repozytorium

Narzędzia deweloperskie:

- **Git** i **GitHub** – kontrola wersji
- **pytest** – testy serwera; **Ruff** – lint i formatowanie kodu Python
- **ESLint** i **Prettier** – lint i formatowanie kodu klienta
- **npm** i **pip** – menedżery pakietów

5.3 Bezpieczeństwo

- Sesja jako token JWT w ciasteczku `HttpOnly` (niedostępnym z JavaScriptu), `SameSite=Lax`
- Rola i aktywność konta odczytywane z bazy przy każdym żądaniu – blokada konta działa natychmiast
- Hasła haszowane algorytmem bcrypt
- Uprawnienia sprawdzane na serwerze; zasób niedostępny dla użytkownika zwraca 404, bez zdradzania jego istnienia
- Wgrywane pliki: biała lista rozszerzeń (bez SVG i HTML), limit 50 MB, losowe nazwy na dysku
- Ankieta i quiz wysyłane do klienta bez klucza odpowiedzi; punkty i wynik liczy serwer
- CORS tylko dla adresu klienta, sekrety wyłącznie w plikach `.env` poza repozytorium

5.4 Licencje

Wszystkie wykorzystane technologie są dostępne na licencjach otwartych, pozwalających na swobodne użycie:

- Next.js, React, Tailwind CSS, shadcn/ui, TanStack Query, TipTap, KaTeX – MIT License
- FastAPI, SQLAlchemy, Alembic, Pydantic, pytest – MIT License
- PostgreSQL – PostgreSQL License (podobna do MIT)

6 Dokumentacja testów

6.1 Cel i zakres testowania

Celem testów jest sprawdzenie reguł systemu (personalizacji, adaptacji, oceniania) oraz uprawnień i poprawności wszystkich endpointów serwera. Testy obejmują wszystkie moduły serwera: konta, kursy i klasy, treść lekcji i pliki, model ucznia oraz quizy, a także dane demonstracyjne.

6.2 Strategia testowania

Poziom	Jednostkowy	Integracyjny
Typ	white-box	black-box HTTP
Narzędzie	pytest	pytest + FastAPI TestClient
Zakres	czyste funkcje reguł	endpointy REST z bazą danych

Testy jednostkowe sprawdzają czyste funkcje (punktacja ankiety, zwijanie materiałów, styl efektywny, aktualizacja wag, propozycja stylu, ocenianie quizu) – bez bazy danych i bez frameworka.

Testy integracyjne wysyłają żądania HTTP do aplikacji przez `TestClient` i korzystają z prawdziwej bazy PostgreSQL (osobna baza `imsu_test`). Schemat bazy testowej jest budowany tymi samymi migracjami Alembic co baza aplikacji, a każdy test działa w transakcji wycofywanej po jego zakończeniu, więc testy są od siebie niezależne. Wspólne dane (uczeń, nauczyciel, administrator, kurs z lekcjami) dostarczają fixture'y pytest.

Interfejs użytkownika sprawdzany jest ręcznie według scenariusza na kontach demonstracyjnych; kod klienta przechodzi lint (ESLint), kontrolę formatowania (Prettier) i build produkcyjny.

6.3 Środowisko testowe

Narzędzie	Wersja / uwagi
Python	3.12+
pytest	testy jednostkowe i integracyjne
FastAPI TestClient (httpx)	żądania HTTP do aplikacji
PostgreSQL	17, baza `imsu_test` w kontenerze Docker
Alembic	budowa schematu bazy testowej
Ruff	lint i formatowanie kodu serwera

6.4 Przypadki testowe

Poniższa tabela zawiera reprezentatywne przypadki testowe. Kolumna *Typ*: **J** – jednostkowy, **I** – integracyjny (HTTP).

ID	Nazwa	Moduł	Typ	Dane wejściowe / warunki	Oczekiwany wynik
TC-01	Rejestracja ucznia	identity	I	`POST /auth/register`: rola *student*	201; ciasteczko sesji `HttpOnly` ✔
TC-02	Rejestracja jako administrator	identity	I	`POST /auth/register`: rola *admin*	422; konto nie powstaje ✔
TC-03	Blokada konta działa od razu	identity	I	administrator blokuje zalogowanego użytkownika	kolejne żądanie `GET /auth/me`: 401 ✔
TC-04	Katalog kursów ucznia	curriculum	I	`GET /courses`: kursy prywatne i opublikowane	uczeń widzi tylko kursy opublikowane ✔
TC-05	Kurs prywatny przez klasę	curriculum	I	kurs prywatny przypisany do klasy ucznia	uczeń otwiera kurs, lekcję i quiz; po usunięciu z klasy – 404 ✔
TC-06	Cudzy kurs	curriculum	I	nauczyciel otwiera kurs innego nauczyciela	404 (bez zdradzania istnienia kursu) ✔
TC-07	Plik niedozwolonego typu	content	I	`POST /files`: plik `.svg` lub `.exe`	400 `FILE_TYPE_NOT_ALLOWED` ✔
TC-08	Plik innego nauczyciela w lekcji	content	I	`PUT /lessons/{id}/blocks`: załącznik z cudzym plikiem	400 `FILE_NOT_FOUND` ✔
TC-09	Ankieta bez klucza	learning_style	I	`GET /survey/questions`	odpowiedzi bez informacji o formie V/A/K ✔
TC-10	Remis w ankiecie	learning_style	I	tyle samo punktów V i A	styl mieszany `VA` ✔
TC-11	Zwinięte materiały	learning_style	I	uczeń o stylu V otwiera lekcję z materiałami A i K	wszystkie bloki; materiały A i K na liście zwiniętych ✔
TC-12	Uczeń bez ankiety	learning_style	I	`GET /lessons/{id}/personalized-blocks`	403 `SURVEY_REQUIRED` ✔
TC-13	Pierwszy sukces i pierwsza porażka	learning_style	J	kolejne wyniki quizu w tej samej lekcji	liczy się tylko pierwszy sukces i pierwsza porażka; powtórzenia nie zmieniają wag ✔
TC-14	Propozycja zmiany stylu	learning_style	J	wagi V 6 / A 9 / K 2, styl V	propozycja: A ✔
TC-15	Styl ręczny wyłącza propozycje	learning_style	J	jak TC-14, źródło stylu: ręczne	brak propozycji ✔
TC-16	Pytania według stylu	assessment	I	uczeń o stylu V pobiera quiz	pytania wspólne i V, bez klucza odpowiedzi ✔
TC-17	Rozwinięty materiał w quizie	assessment	I	uczeń rozwija materiał innej formy i pobiera quiz ponownie	w quizie pojawia się pytanie o ten materiał ✔

Kontynuacja na następnej stronie...

ID	Nazwa	Moduł	Typ	Dane wejściowe / warunki	Oczekiwany wynik
TC-18	Wielokrotne podejścia	assessment	I	dwa zaliczone podejścia w tej samej lekcji	wagi zmieniają się tylko po pierwszym ✔
TC-19	Pytanie spoza stylu	assessment	I	odpowiedź na pytanie formy, której uczeń nie widzi	400 `QUIZ_ANSWER_MISMATCH` ✔
TC-20	Idempotentny seed	demo	I	dwukrotne uruchomienie seeda	brak duplikatów kursów, lekcji i klasy ✔

6.5 Scenariusze testowe

6.5.1 ST-01: Od ankiety do adaptacji

Cel: sprawdzenie pełnej ścieżki ucznia.

1. Uczeń bez ankiety próbuje otworzyć lekcję (TC-12) – 403 `SURVEY_REQUIRED`.
2. Uczeń wypełnia ankietę – powstaje profil ze stylem i wagami (TC-09, TC-10).
3. Uczeń otwiera lekcję – materiały innych stylów są zwinięte (TC-11).
4. Uczeń rozwija materiał i rozwiązuje quiz – pytania odpowiadają stylowi efektywnemu (TC-16, TC-17).
5. Wynik quizu zmienia wagi; kolejne podejście już ich nie zmienia (TC-18).

6.5.2 ST-02: Propozycja zmiany stylu

Cel: sprawdzenie reakcji systemu na wyniki ucznia.

1. Wagi ucznia o stylu V spadają po niezaliczonych quizach do V 7 / A 8 / K 3.
2. Profil ucznia zawiera propozycję zmiany stylu na A z uzasadnieniem (TC-14).
3. Uczeń odrzuca propozycję – nie pojawia się ona ponownie.
4. Uczeń wybiera styl ręcznie – propozycje są wyłączone (TC-15).

6.5.3 ST-03: Dostęp do kursów przez klasy

Cel: sprawdzenie widoczności kursów prywatnych.

1. Nauczyciel tworzy klasę, dodaje ucznia i przypisuje kurs prywatny.
2. Uczeń nie widzi kursu w katalogu (TC-04), ale otwiera go przez klasę (TC-05).
3. Uczeń spoza klasy i inny nauczyciel otrzymują 404 (TC-06).
4. Po usunięciu ucznia z klasy kurs przestaje być dla niego dostępny.

6.6 Podsumowanie pokrycia testami

Moduł	Jednostkowe	Integracyjne	Łącznie
`identity`	6	42	48
`curriculum`	–	64	64
`content`	2	23	25
`learning_style`	23	42	65
`assessment`	6	15	21
`demo`	–	10	10
`core` (błędy, stan serwera)	–	5	5
Razem	**37**	**201**	**238**

Tabela podaje liczbę funkcji testowych. Część testów jest sparametryzowana (ten sam test dla wielu danych wejściowych), dlatego pytest wykonuje łącznie **349** przypadków testowych.

7 Instrukcja uruchomienia aplikacji lokalnie

Baza danych działa w kontenerze Docker, a serwer i klient są uruchamiane lokalnie. Polecenia podano dla systemu Windows (wiersz poleceń `cmd`); każde polecenie należy wykonać osobno.

7.1 Wymagania

- **Docker Desktop** – baza danych PostgreSQL (`https://www.docker.com/products/docker-desktop/`)
- **Python 3.12** lub nowszy – serwer (`https://www.python.org/downloads/`)
- **Node.js 20** lub nowszy wraz z **npm** – klient (`https://nodejs.org/`)
- **Git** – pobranie repozytorium (`https://git-scm.com/downloads`)

Sprawdzenie zainstalowanych narzędzi:

```
docker --version
python --version
node --version
git --version
```

7.2 Krok 1: Pobranie repozytorium

```
git clone https://github.com/PatrykKul/aplikacjaIMSU.git
cd aplikacjaIMSU
```

7.3 Krok 2: Zmienne środowiskowe

Projekt korzysta z trzech plików ze zmiennymi środowiskowymi. Każdy powstaje przez skopiowanie dołączonego wzoru (pliki `.env` nie trafiają do repozytorium):

```
copy .env.example .env
copy backend\.env.example backend\.env
copy frontend\.env.example frontend\.env.local
```

Baza danych (`.env` w katalogu głównym):

```
POSTGRES_USER=imsu
POSTGRES_PASSWORD=imsu
POSTGRES_DB=imsu
POSTGRES_PORT=5433
```

Serwer (`backend\.env`):

```
DATABASE_URL=postgresql+psycopg://imsu:imsu@localhost:5433/imsu
TEST_DATABASE_URL=postgresql+psycopg://imsu:imsu@localhost:5433/imsu_test
CORS_ORIGINS=["http://localhost:3001"]
JWT_SECRET=change_me_to_a_random_string_of_at_least_32_chars
UPLOAD_DIR=uploads
```

Klient (`frontend\.env.local`):

```
NEXT_PUBLIC_API_URL=http://localhost:8001
```

Klucz JWT

`JWT_SECRET` musi mieć co najmniej 32 znaki – bez niego serwer się nie uruchomi. Wartość ze wzoru wystarcza do uruchomienia lokalnego; własny losowy klucz można wygenerować poleceniem `python -c "import secrets; print(secrets.token_hex(32))"`.

7.4 Krok 3: Baza danych

Uruchom Docker Desktop, a następnie w katalogu głównym repozytorium:

```
docker compose up -d
```

Polecenie uruchamia PostgreSQL 17 na porcie **5433**. Przy pierwszym starcie kontener tworzy dwie bazy: `imsu` (aplikacja) i `imsu_test` (testy).

7.5 Krok 4: Serwer

Instalacja zależności w środowisku wirtualnym (tylko przy pierwszym uruchomieniu):

```
cd backend
python -m venv venv
venv\Scripts\python.exe -m pip install -r requirements.txt
```

Utworzenie tabel i wstawienie pytań ankiety (migracje), a następnie wczytanie danych demonstracyjnych:

```
venv\Scripts\python.exe -m alembic upgrade head
venv\Scripts\python.exe -m app.demo.seed
```

Uruchomienie serwera:

```
venv\Scripts\python.exe -m uvicorn app.main:app --reload --reload-dir app --port
    8001
```

Serwer działa pod adresem `http://localhost:8001`, a dokumentacja API (Swagger) pod `http://localhost:8001/docs`.

7.6 Krok 5: Klient

W drugim oknie wiersza poleceń, z katalogu głównego repozytorium:

```
cd frontend
npm install
npm run dev
```

Aplikacja jest dostępna pod adresem `http://localhost:3001`. Wersję produkcyjną klienta uruchamia się poleceniami `npm run build` i `npm run start` (ten sam port).

7.7 Krok 6: Weryfikacja

1. Adres `http://localhost:8001/health` zwraca odpowiedź:
   `{"status": "ok", "database": "ok"}`

2. Pod adresem `http://localhost:3001` widać stronę startową z przyciskami „Zaloguj się” i „Załóż konto”

3. Logowanie na konto demonstracyjne działa (hasło wszystkich kont: `Demo123!`):

Konto	**Rola**	**Stan**
`nauczyciel.demo@imsu.pl`	nauczyciel	3 kursy z materiałami i quizami, klasa „Klasa 7a”
`uczen.nowy@imsu.pl`	uczeń	bez ankiety (pokaz wymuszenia ankiety)
`uczen.demo@imsu.pl`	uczeń	historia quizów, propozycja zmiany stylu
`admin.demo@imsu.pl`	administrator	zarządzanie kontami

Tabela 4: Konta demonstracyjne

Stan startowy obu kont uczniów przywraca polecenie (z katalogu `backend`):

```
venv\Scripts\python.exe -m app.demo.seed --reset-students
```

Własne konto administratora tworzy się w konsoli (hasło jest podawane interaktywnie):

```
venv\Scripts\python.exe -m app.modules.identity.cli create-admin --email
    admin@example.com --name "Administrator"
```

7.8 Uruchomienie testów i kontrola kodu

Serwer (katalog `backend`, baza danych musi działać):

```
venv\Scripts\python.exe -m pytest
venv\Scripts\python.exe -m ruff check .
venv\Scripts\python.exe -m ruff format --check .
```

Klient (katalog `frontend`):

```
npm run lint
npm run format:check
npm run build
```

7.9 Rozwiązywanie problemów

7.9.1 `/health` zwraca `"database": "error"`

Baza danych nie działa. Uruchom Docker Desktop i ponownie wykonaj `docker compose up -d` w katalogu głównym repozytorium (konieczne także po restarcie komputera).

7.9.2 Serwer nie startuje: błąd `jwt_secret`

W pliku `backend\.env` brakuje zmiennej `JWT_SECRET` lub ma ona mniej niż 32 znaki.

7.9.3 Port 3001, 8001 lub 5433 jest zajęty

Błąd `EADDRINUSE` przy `npm run dev` oznacza zwykle, że klient już działa w innym oknie. Port bazy można zmienić zmienną `POSTGRES_PORT` (i odpowiednio adresy w `backend\.env`).

7.9.4 Brak pytań ankiety lub kursów demonstracyjnych

Wykonaj migracje (`alembic upgrade head`), a potem seed (`app.demo.seed`). Seed można uruchamiać wielokrotnie – dokłada tylko brakujące dane.

7.10 Struktura projektu

```
aplikacjaIMSU/
  docker-compose.yml        # baza danych PostgreSQL
  docker/                   # skrypt tworzacy baze testowa
  backend/                  # serwer - FastAPI
    requirements.txt        # zaleznosci Python
    alembic/                # migracje bazy danych
    app/
      main.py               # punkt wejscia aplikacji
      core/                 # konfiguracja, sesja bazy, bledy
      modules/
        identity/           # konta, role, logowanie
        curriculum/         # kursy, lekcje, klasy
        content/            # bloki lekcji, pliki
        learning_style/     # ankieta, profil, personalizacja, adaptacja
```

```
      assessment/          # quiz lekcji, ocenianie
    demo/                  # dane demonstracyjne (seed, materialy)
  tests/                   # testy pytest
frontend/                  # klient - Next.js
  src/
    app/                   # trasy (strony)
    features/              # funkcje: auth, courses, teach, classes,
                           #   content, learning-style, assessment...
    shared/                # komponenty UI, klient API, uklad strony
docs/                      # dokumentacja projektu
```

7.11 Przydatne polecenia

```
# Baza danych (katalog glowny)
docker compose up -d                                    # start bazy
docker compose down                                     # zatrzymanie bazy

# Serwer (katalog backend)
venv\Scripts\python.exe -m alembic upgrade head         # migracje
venv\Scripts\python.exe -m app.demo.seed                # dane demo
venv\Scripts\python.exe -m uvicorn app.main:app --port 8001   # start
venv\Scripts\python.exe -m pytest                       # testy

# Klient (katalog frontend)
npm install                                             # zaleznosci
npm run dev                                             # start (deweloperski)
npm run build                                           # build produkcyjny
npm run start                                           # start (produkcyjny)
```